

1. Mini-Batches 🚗

Problem: Standard gradient descent (using the entire dataset) is slow, especially when the dataset is large.

- In Reinforcement Learning, we might have a Replay Buffer with 100,000 or 1,000,000 experiences.
- Calculating the gradient over *all* these examples every single step is computationally expensive.

Solution: Use **Mini-Batch Learning**.

- Instead of using the full 100,000 examples, pick a random subset (e.g., **1,000 examples**) for each training step.
- **Effect:**
 - **Speed:** Each step is much faster (computing gradients for 1,000 items is instant compared to 100,000).
 - **Noise:** The path to the minimum is "noisier" (zig-zagging) because each batch is just an approximation of the whole.
 - **Overall:** It converges *much* faster in terms of wall-clock time.

In RL Context:

Instead of training on the *entire* history of the robot's life every time, we just grab a random sample of recent memories (e.g., 64 or 128 experiences) and train on those.

2. Soft Updates ☁️

Problem: In the original algorithm, we completely replaced the old network weights (W) with the new weights (W_{new}) at every step ($Q = Q_{new}$).

- If Q_{new} is a bad or noisy estimate (which happens often in RL), overwriting the old network completely can cause the agent to "forget" what it learned or make the training unstable/diverge.

Solution: Use **Soft Updates** (Polyak Averaging).

Instead of a 100% replacement, we slowly blend the new weights into the old ones.

Formula:

$$W \leftarrow \tau W_{new} + (1 - \tau)W$$

$$B \leftarrow \tau B_{new} + (1 - \tau)B$$

- τ (**Tau**): A small number (e.g., **0.001** or **0.01**).

- If $\tau = 0.01$: We keep **99%** of the old weights and mix in just **1%** of the new learned weights.

Effect:

- The network changes **gradually**.
 - This prevents catastrophic forgetting and makes the training process **much more stable** and reliable.
-

Summary of the Final DQN Algorithm

1. **Experience:** Agent plays and stores tuples (s, a, R, s') in Replay Buffer.
2. **Mini-Batch:** Sample a small batch (e.g., 64) from the buffer.
3. **Train:** Calculate targets (Y) and update the **Local Network** weights (W_{new}) using Gradient Descent.
4. **Soft Update:** Update the **Target Network** weights (W) by blending in a tiny amount of W_{new} ($\tau W_{new} + (1 - \tau)W$).